ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA (SEDE DI BOLOGNA)

Anno Accademico 2024/2025

RELAZIONE DI FINE TIROCINIO CURRICULARE

svolto dallo studente:

Tommaso Ripanti (matricola nr 1071002)

iscritto al Corso di Studio in

L-8 Ingegneria informatica (codice CdS: 9254)

presso:

Indici Opponibili

sul seguente argomento:

Videogame per il Museo del Mare di Trieste

Studente:

Tommaso Ripanti

(firma)

Tutor Accademico:

Prof. Paolo Bellavista

(firma per approvazione della relazione finale)

Referente Struttura Ospitante:

(Samuele Bertani, Business developer e Innovation Manager)

(firma per approvazione della relazione finale)

Indice

INTRODUZIONE 4

L'ESPERIENZA 6

L'azienda 6

Le tecnologie 8

Discord 8

Notion 9

Cocos Creator 9

Figma 12

Attività svolte 12

Meccaniche di gioco 15

Asset replacement 25

Animation 26

Polish 26

CONCLUSIONE 28

SITOGRAFIA 30

INTRODUZIONE

Ho scelto di inoltrarmi in questo percorso di tirocinio per comprendere al meglio i miei interessi e prendere consapevolezza del mondo professionale. Infatti, questa opportunità ha rafforzato la mia propensione verso il *game development*, oltre a far luce sulle dinamiche e la struttura di una piccola realtà imprenditoriale italiana. Questo e non solo, in quanto l'esperienza mi ha incoraggiato nella difficile sfida di formarmi e occupare un posto di lavoro nel campo videoludico in Italia, dato che il mercato del settore è ancora di nicchia nel nostro paese. Tuttavia, i finanziamenti, gli eventi e le imprese aumentano, seguendo una tendenza promettente. Riporto di seguito i dati raccolti da IIDEA[1]: <<Il comparto, che nel 2012 contava 48 imprese e un fatturato di circa 20 milioni di euro, nel 2024 ha superato i 200 operatori e raggiunto un fatturato stimato tra 180 e 200 milioni di euro, con un incremento del 36% rispetto al 2022>>. Attualmente, non ci sono grandi *software house* che producono videogiochi, piuttosto il territorio è popolato da entità *indie* che si presentano al pubblico nei diversi raduni nazionali, quali il FirstPlayable[2] a Firenze, lo Svilupparty[3] a Bologna, Cotoletta8bit[4] a Milano e altre fiere del fumetto e dei giochi come la Milan Games Week[5] a Milano, il Lucca Comics & Games[6] a Lucca, il Romics[7] a Roma e tanti altri in varie regioni.

Spinto dal sogno di dare il mio contributo all'Italia dei videogiochi, ho preso coraggio e mi sono fatto avanti contattando l'azienda per posta elettronica. Ho tenuto dunque diversi colloqui con il referente aziendale, *business developer* e

[1]Italian Interactive Digital Entertainment Association è un'associazione con l'obiettivo di rappresentare e unire il settore dei videogiochi in Italia. Si impegna << in attività come la pubblicazione di rapporti di mercato, rilevazioni sugli operatori del settore, collaborazioni con istituzioni pubbliche e la promozione della cultura videoludica attraverso eventi e iniziative sul territorio >>.

[2]Evento tenuto a Firenze annualmente con *talk* ed esposizioni dedicati agli sviluppatori di videogiochi italiani. Organizzato da IIDEA e Toscana Film Commission – Fondazione Sistema Toscana, in collaborazione con Agenzia ICE.

[3]Evento bolognese simile al FirstPlayable, organizzato dall'Associazione Italian Party of Indie Developers (IPID).

[4]Evento milanese simile al FirstPlayable con anche giochi di ruolo e da tavolo, organizzato da Francesco Toniolo, Mattia La Licata e Fabio Putortì.

[5]Evento internazionale e annuale dedicato ai videogiochi, ai fumetti e alla cultura pop, che si tiene a Milano ed è organizzato da Fiera Milano e Fandango Club.

[6]Uno dei più grandi festival europei dedicati al fumetto, al gioco, al videogioco, al cinema e all'illustrazione. Si tiene ogni anno a Lucca ed è organizzata da Lucca Crea.

[7]Festival internazionale semiannuale dedicato al fumetto, all'animazione, ai giochi, al cinema e al cosplay. Si tiene a Roma ed è organizzato da ISI.urb e FieraRoma.

innovation manager della società Samuele Bertani, e affrontato una piccola prova di problem solving. Inoltre, per la valutazione della mia richiesta sono stato sollecitato a fornire tre documenti di presentazione, quali il curriculum vitae, il portfolio, e una lettera motivazionale.

L'ESPERIENZA

L'azienda

L'azienda che ha accettato la mia candidatura, di cui sopra, è la società cooperativa Indici Opponibili, con sede legale nella città di Bologna. In veste di cooperativa, il fine ultimo dell'impresa non è lo scopo di lucro, bensì quello del benessere dei soci in termini economici, sociali e culturali. I soci ricevono uno stipendio in base alle loro funzioni e ai loro ruoli aziendali e gli utili che nascono dall'esercizio dell'attività sono per di più reinvestiti nella stessa. Anche se, per maggior dettaglio, il 3% dei ricavi di tali società è destinato ai fondi mutualistici, pensati per supportare le cooperative esistenti e promuovere le nuove. Questi ultimi sono gestiti da associazioni a carattere nazionale, come Legacoop, a cui è affiliata Indici. Ultima notevole differenza che la distingue dalle altre società è il principio "una testa, un voto", che porta alla partecipazione all'assemblea da parte dei soci senza distinzione delle quote di capitale possedute. Questo e altro, sono i punti che risaltano nel loro manifesto, di cui ho preso visione prima di iniziare il tirocinio.

Per quel che riguarda l'oggetto della loro attività, si mostrano al pubblico con la denominazione di *Digital agency*: termine che indica le aziende che offrono servizi ospitati su dispositivi digitali. Un'etichetta eccessivamente generica che per loro si coniuga concretamente nello sviluppo di videogiochi, per di più *serious / applied game*[8] per il mercato B2B[9], nonostante abbiano già mosso i primi passi verso il B2C[10] con la pubblicazione del loro *desktop*-game Handmancers sullo store online Steam, sotto il loro brand dedicato 58Blades. Si applicano anche nello sviluppo di siti web provvisti di *gamification*[11], tramite meccanismi di *achievements*[12] e *scrollytelling*[13], per coinvolgere

[8]Si intende un gioco progettato con uno scopo principale diverso dal puro intrattenimento, come può essere l'educazione, la salute o l'arte.
[9]Acronimo di Bussines To Bussines, a indicare un'attività che realizza prodotti o servizi per le aziende.
[10]Acronimo di Businnes To Consumer, come B2B ma per i consumatori finali che non hanno scopo professionale.
[11]Uso di elementi di gioco in contesti non ludici.
[12]Meccanismi che premiano gli utenti per aver raggiunto determinati traguardi o completato specifiche attività.
[13]Tecnica narrativa per cui gli elementi visivi e testuali di una pagina cambiano man mano che l'utente scorre verso il basso

maggiormente l'utente e mantenere alta la sua attenzione. In più, si occupano di *motion graphics*[14] e di politiche e strategie per migliorare il SEO[15] di una *landing page*[16], così come di *social media strategy* e vetrine per annunci di *call-to-action*[17] e *fundraising*[18].

In riferimento alle dinamiche aziendali, il loro processo di sviluppo si ispira al modello dell'*extreme programming* e della prototipazione agile. Questo vuol dire che le fasi di analisi, progettazione, implementazione e collaudo del prodotto sono rapide, sovrapposte fra loro e ripetute ciclicamente in modo incrementale, cioè perfezionando in tal modo il prodotto in lavorazione. Dato che alcuni requisiti e funzionalità del servizio rimangono vaghi e in continua evoluzione, è fondamentale che il loro assetto lavorativo ruoti intorno al dialogo tra i membri, suddivisi in piccoli *team* a cui viene affidato un progetto ciascuno. In questo scenario, i progressi e le problematiche riscontrate vanno comunicate tempestivamente, in modo che il resto del gruppo possa reagire e adattarsi alle evoluzioni in modo agile. Nella mia esperienza, tutto ciò si è tradotto sia positivamente, perché ho potuto seguire il processo da capo a piedi, interfacciandomi settimanalmente col progettista e l'altro programmatore, sia con dei punti deboli, visto che alcune attività importanti, ma secondarie per la sola consegna del prodotto, sono state tralasciate, come la documentazione e il documento progettuale, privo di diagramma dei *package* e di quello delle classi. Questo modus operandi ha quindi il pregio di velocizzare la produzione nel breve termine, ma, secondo quello che ho studiato, riduce la riusabilità dei componenti e rende più difficile un'eventuale manutenzione o estensione del contenuto digitale.

In questo contesto, quasi tutte le mie relazioni professionali sono avvenute da remoto, salvo per una giornata in cui, senza obbligo, ho approfittato della

---

[14]Tecnica per cui elementi grafici come testo, forme, icone e illustrazioni si muovono con traslazioni e rotazioni brevi e rapide. Il focus principale di queste è posto sull'efficacia comunicativa.

[15]Acronimo di *Search Engine Optimization*, a indicare l'insieme di tecniche e strategie utilizzate per migliorare la visibilità di un sito web nei risultati di un motore di ricerca.

[16]Pagina web che il visitatore raggiunge dopo aver cliccato su un link. Ottimizzata per una specifica parola chiave per attrarre i motori di ricerca. Ha lo scopo di trasformare gli utenti in clienti.

[17]Elemento di una pagina web, o altro contenuto di marketing che invita gli utenti a compiere una specifica azione.

[18]Processo di raccolta fondi per sostenere progetti, organizzazioni o cause specifiche.

presenza di alcuni colleghi in sede per conoscere dal vivo la loro realtà. Invero, il lavoro in Indici è *full remote* e *part-time*, di modo che il personale possa guadagnarci in termini di vita extra-lavorativa. Per concretizzare questa scelta, Indici ha deciso di appoggiarsi alla piattaforma di messaggistica e *voice over ip*[19] Discord, aprendo un suo "server" con molteplici canali di interazione per le *call* di gruppo, le riunioni generali, le *chat* con i clienti e quelle finalizzate al coordinamento interno e altrettante per la condivisione di contenuti e articoli sulle notizie di attualità *tech,* eventi di rilievo sul territorio, corsi di formazione e molto ancora. Da qui si nota come i soci siano attenti al benessere collettivo, tanto da organizzare aperitivi e incontri di persona, per rafforzare le conoscenze, l'identità dell'azienda e coltivare un sano ambiente di lavoro.

Nell'elenco esemplificativo sopra dispiegato, ho accennato ai loro ritrovi quotidiani a cui partecipano tutti i componenti dell'azienda. Questi sono gli *stand-up meeting* di Indici, organizzati all'inizio di ogni giornata, con lo scopo di aggiornare ogni parte sui progetti correnti e sul loro stato di avanzamento. Dopodiché, ciascun lavoratore si disloca nel canale opportuno per proseguire con i *task* del giorno assieme al suo gruppo momentaneo di appartenenza. Anche io sono stato invitato a condividere con loro questo rituale mattutino e, quando possibile, ho ascoltato la scaletta di interventi, moderati dal direttore creativo della cooperativa Andrea Montuschi. Per mia fortuna però, ho potuto, per la maggior parte del tirocinio, organizzare autonomamente il mio tempo e le mie attività, ad eccezion fatta delle *call* di verifica dei progressi, programmate in congiunzione con i miei supervisori e colleghi di progetto: il *game programmer* Lorenzo Dezi e il *project manager* e *game designer* Gianmarco Albertini.

Le tecnologie

Illustro brevemente le tecnologie di cui ho fatto uso durante il percorso di tirocinio.

Discord

Discord è una piattaforma di messaggistica istantanea e VoIP che offre servizi di comunicazione digitale tramite testo, voce e video. Nello specifico,

[19]Tecnologia che consente di effettuare chiamate vocali utilizzando una connessione Internet anziché le tradizionali linee telefoniche.

l'interazione può essere privata tra due utenti o avvenire in comunità virtuali chiamate "server", composti da uno o più canali di comunicazione e accessibili via link. Da sapere che l'applicazione è distribuita sotto licenza proprietaria e i diritti patrimoniali appartengono alla Discord Inc.

Notion

Notion è un'applicazione di produttività e gestione delle informazioni che consente agli utenti di prendere appunti, pianificare e monitorare progetti, quindi impostare scadenze e obiettivi. In aggiunta, permette di creare e gestire database personalizzati e wiki per la condivisione delle conoscenze. Tutto ciò, con la possibilità di collaborare, anche in *real time*, con i membri del *team* su *workspace*[20] condivisi. Notion è stato sviluppato dalla Notion Labs Inc. ed è distribuito sotto licenza proprietaria con un modello di abbonamento per l'utilizzo freemium[21].

Cocos Creator

Cocos Creator è un ambiente di sviluppo per i videogiochi, *open source*[22] sotto la licenza MIT[23] e gratuito, rilasciato dalla Xiamen Yaji Company. Questo *engine* viene utilizzato ad oggi come strumento per il *game development* sia in 2D sia in 3D, oltre che per le applicazioni di realtà virtuale ed estesa. Inoltre, come gli altri software concorrenti, quali Godot, Unity, Unreal ecc., è un motore per lo sviluppo *cross-platform*[24]. Nonostante ciò, come sarà più chiaro a lettura avanzata, ci si è avvalsi di un *framework* aggiuntivo per rendere il gioco portabile sul sistema Windows. Dunque, le due principali motivazioni che hanno favorito la scelta di quest'ultima tecnologia sono per di più la competenza preesistente nell'uso di questo ambiente da parte dei membri del *team* e le convenienze derivanti dal software libero.

---

[20]Ambiente virtuale che consente agli utenti di gestire e organizzare le proprie attività lavorative.

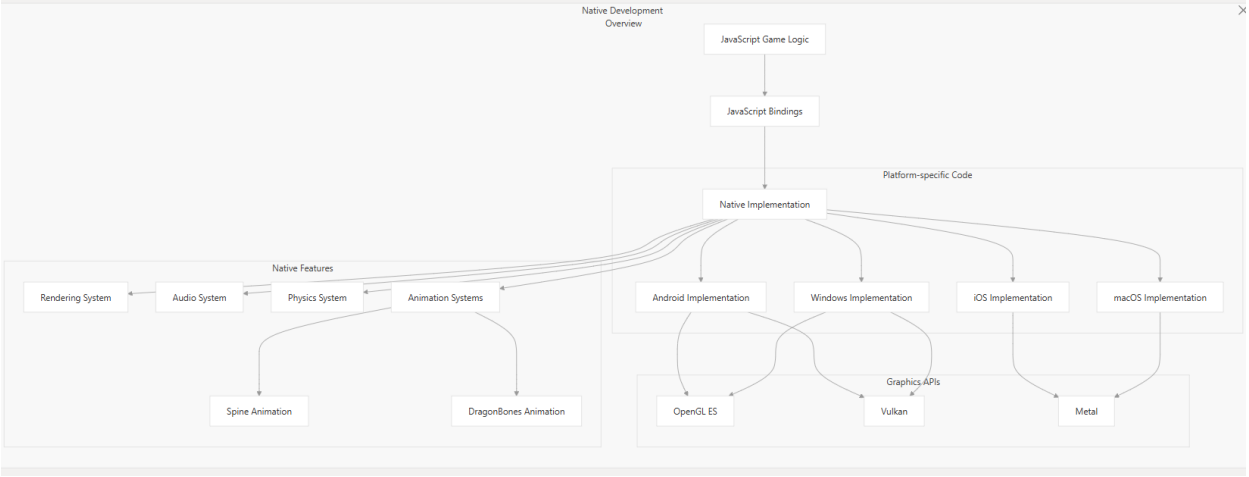
[21]Strategia di marketing che offre una versione base di un prodotto o servizio gratuitamente, mentre le funzionalità aggiuntive o avanzate sono disponibili a pagamento.

[22]Software con codice sorgente disponibile pubblicamente e che può essere utilizzato, modificato e distribuito da chiunque.

[23]Licenza permissiva che consente agli utenti di utilizzare, copiare, modificare, fondere, pubblicare, distribuire, sublicenziare e vendere copie del software con l'unica condizione di includere licenza originale e copyright in tutte le copie del software.

[24]Capacità di un software di funzionare su diverse piattaforme senza necessità di modifiche significative.

I moduli typescript sviluppati dal programmatore e le altre risorse sono gestite in esecuzione dal Cocos *engine*, cioè un *run-time environment* che interpreta il codice javascript e che tramite i suoi meccanismi interni si fa carico dell'interoperabilità con la piattaforma sottostante. Non solo questo, perché nella sua struttura a più *tier* sono previsti moduli che si occupano di far funzionare diversi sistemi integrati, come quelli dell'animazione, della fisica, oltre al sistema particellare, a quello di gestione della UI e per la *preview* istantanea. Mostro qui sotto uno schema dei livelli:



I concetti chiave in Cocos sono:

1. le scene, che rappresentano un insieme di elementi che compongono il mondo di gioco, anche detti *game object;*
2. i nodi, che sono gli elementi della scena e fungono da contenitori per i componenti. Ogni nodo può avere più figli e un solo nodo padre, creando una struttura gerarchica chiamata *node tree;*
3. i componenti, che sono i moduli associabili al nodo, così da equipaggiarlo con funzionalità specifiche, come il rendering, la fisica, l'animazione ed altri comportamenti personalizzati via *script*. Contengono perciò dati e logica relativi ad una particolare caratteristica;
4. l'architettura *entity-component-system* (ECS), che propone il nodo (*entity*) come un contenitore di più componenti (*component*), facilmente aggiungibili e rimovibili e che modellano i dati del dominio. Infine i sistemi (*system*), si preoccupano di implementare la logica applicativa, iterando attraverso le entità con determinati componenti. Ad esempio, un

sistema di movimento potrebbe iterare attraverso tutte le entità che hanno un componente di posizione e aggiornare le loro coordinate in base all'input dell'utente. Come riferisce una fonte di Wikipedia: <<ECS follows the principle of composition over inheritance, meaning that every entity is defined not by a type hierarchy, but by the components that are associated with it>>. Questo accresce fattori di qualità interni, quali modularità, manutenibilità e riusabilità del codice;

5. due oggetti *singleton*, Game e Director, si occupano della gestione del *game loop* e delle transizioni di scena, oltre a processare gli *input* dell'utente e altri eventi chiave. In pratica, Il *game loop* coordina il *life cycle* dei nodi e propaga un cambio di stato, come creazione, attivazione, disattivazione, distruzione ed altre fasi, a ciascuno di essi così da ripercuotere l'evento in ogni componente. Per farlo, si usa un approccio *deep-first* o *cascade*, per cui viene prima processato il nodo radice e, a seguire, tutti i suoi figli. Lo schema continua uguale per ogni nodo genitore che si incontra nella gerarchia, navigando prima in profondità fino alle foglie della struttura ad albero e solo dopo passando al secondo nodo fratello, in cui ci si imbatte mentre si risale la piramide;
6. il sistema degli eventi, che si fonda sul *pattern publisher-subscriber*, permettendo la comunicazione tra più componenti. In breve, un componente è in grado di ricevere un segnale solo previa sottoscrizione che lega un evento ad una funzione di *callback*. Chi vuole trasmettere un'informazione invece, può farlo emettendo un evento. In tal caso, seguendo una politica di tipo non *deferred*, vengono invocate nell'immediato le *callback*, nell'ordine di sottoscrizione, senza rimandare l'esecuzione in un secondo momento.

Faccio qui vedere un'immagine riassuntiva del *node system*:

Figma

Figma è una web app per il *design* collaborativo di interfacce utente. Questo significa che offre strumenti di progettazione per realizzare *wireframe*[25] e *mockup*[26] con grafiche vettoriali[27], così come prototipi interattivi per valutare la UX[28], garantendo a più *designer* di lavorare simultaneamente. La piattaforma è sotto una licenza proprietaria e viene rilasciata in diverse modalità in base al tipo di piano, gratuito o a pagamento con più o meno *features*.

Attività svolte

L'obiettivo del progetto, in cui sono stato coinvolto, consisteva nello sviluppo di un videogioco commissionato da Hibou ma da installare in un *totem* al termine del percorso del Museo Del Mare di Trieste. Il prodotto è un *serious game*, formato da due *gameplay* interdipendenti: uno pseudo *platformer* con prospettiva dall'alto e un *quiz game*.

[25]Bozza dell'interfaccia grafica di una pagina web o applicazione.

[26]Rappresentazione visiva ad alta fedeltà che mostra come apparirà una applicazione graficamente a fine sviluppo.

[27]Immagini salvate internamente come equazioni matematiche di punti, linee, curve e poligoni. A differenza delle *raster,* queste non perdono qualità ridimensionandole.

[28]Acronimo di User Experience, a indicare l'insieme delle percezioni che un utente ha quando interagisce con un prodotto.

Il primo dei quali, cioè il principale, prevede un personaggio comandato dal giocatore mediante *tap* sullo schermo, che lo direziona verso destra o verso sinistra. L'obiettivo del giocatore è proseguire lungo il percorso composto da piattaforme e suddiviso in tre livelli, sfuggendo da uno squalo, mentre supera ostacoli e risponde a domande, poste al raggiungimento di specifiche caselle. Ne dettaglio qui alcuni aspetti:

1. lo squalo insegue il protagonista distanziandolo al massimo di tre piattaforme di intermezzo. Ne consegue che potrà variare la sua velocità, così da adeguarsi a quella del giocatore al bisogno;
2. gli ostacoli sono di due categorie: ostacoli mobili e immobili, gli ultimi dei quali si biforcano in ulteriori due tipologie. I primi compaiono e scompaiono secondo un ritmo fissato e bloccano il passaggio quando visibili. Se il giocatore li colpisce, allora il cronometro che misura la permanenza nello stato visibile si azzera, ritardandone la scomparsa. I secondi bloccano il passaggio fino alla loro distruzione, resa possibile dallo scontro con il giocatore. Per quelli di primo grado è sufficiente un colpo, mentre ne servono due per quelli del secondo;
3. l'atterraggio su determinate piattaforme avvia il secondo *gameplay* di quiz. Una di queste è l'ultima casella del livello, che è speciale in quanto permette al giocatore di transitare al livello successivo o allo stato di vittoria;
4. per ogni piattaforma raggiunta e ostacolo danneggiato e superato, si guadagnano dei punti che accrescono il punteggio globale.

Il secondo *gameplay*, di contro, prevede un'interrogazione con una domanda e tre risposte, tra cui solo una corretta. Il quesito viene accompagnato da un'immagine che lo rappresenta, con la possibilità di ingrandirla in un *popup,* e, se presente, da una curiosità relativa che appare una volta data la risposta giusta. Ad ogni errore commesso nel rispondere, si riducono i punti potenzialmente ottenibili a fine *quiz*.

Per implementarlo, si è deciso di costruirlo sopra una base già pronta di un vecchio gioco dell'azienda, così da poter riusare molte delle sue parti e ridurre i tempi.

Lascio qui diverse immagini che descrivono parti del gioco. La prima rappresenta una porzione degli pseudo diagrammi UML delle classi, i quali diagrammi ho tracciato come parte della documentazione di fine progetto e pensato per possibili futuri sviluppi. Le altre, a seguire, sono dei frammenti del *mockup* di gioco preso dal *workspace* di Figma.

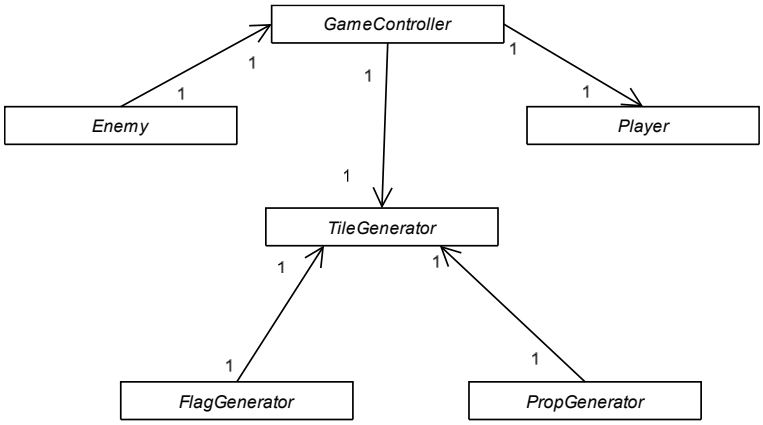


Diagramma delle classi parziale

Mockup di un livello

Mockup del popup nel quiz

Mockup di un menù di fine livello

Mockup del menù di vittoria

Meccaniche di gioco

Nella prima fase del tirocinio, mi è stato innanzitutto chiesto di ambientarmi con i nuovi strumenti, per poi analizzare un loro progetto preesistente da cui il nuovo gioco ha preso forma. E così ho fatto. In questa sede, comunque, scelgo di puntualizzare i soli aspetti che sono parsi a me interessanti: il concetto di classe ed ereditarietà nel mondo ECMAscript[29] e lo *shaping* di un particolare oggetto *singleton*[30].

Entrambi gli argomenti si sono rivelati assai formativi ma il primo in modo significativo. In effetti, il mio corso di Tecnologie Web mi aveva fornito una buona infarinatura del linguaggio Javascript, e aveva ribadito con determinazione l'assenza della classe come vera e propria entità della tecnologia, tanto da etichettare il linguaggio *object-based* ma assolutamente non *class-based*. Tutto più che corretto, ma le argomentazioni mi avevano persuaso di non poter sfruttare i costrutti e i principi a cui tanto ero affezionato, provenendo dal mondo Java: le classi come *template* di oggetti e i principi di ereditarietà e polimorfismo. Al contrario, l'analisi del vecchio codice ha

[29]ECMAScript è uno standard di scripting che costituisce la base di linguaggi di programmazione come JavaScript.
[30]Design pattern utilizzato in programmazione per garantire che una classe abbia una sola istanza e che questa istanza sia accessibile globalmente.

confermato l'esistenza di queste astrazioni e delle parole chiave *class* ed *extends* a partire da ES6[31], nonostante siano costruite al di sopra di un fondamento diverso: il *prototyping*. Infatti in Javascript, ogni oggetto, che ben rappresenta una struttura dati, non solo è una collezione di campi definiti dal programmatore, ma presenta delle proprietà di *default* tra cui `__proto__` che contiene il riferimento ad un altro oggetto, suo prototipo. Da quanto detto, se ne deduce una *prototype chain* che regredisce fino all'entità *built-in* Object. Considerando, inoltre, che Javascript si basa su *first-class function*[32], quindi su funzioni assimilabili ad oggetti, anche esse possiedono un proprio prototipo. In più, la funzione costruttore, invocata con la parola chiave *new*, assegna all'oggetto allocato il suo stesso *prototype*, oltre a gestire le altre proprietà, eventualmente allocandone di nuove. Ne deriva che, dietro l'apparenza, la definizione di una classe non è altro che quella di un semplice costruttore. Ed estendere una classe vuol dire, sottobanco, puntare ad un altro costruttore che funge da prototipo. In questo contesto, se viene invocato un metodo o acceduta una variabile non direttamente definito in quell'oggetto ma in uno dei suoi *prototype* da cui discende, allora il *run-time environment* che interpreta il codice, quale il browser ad esempio, ricercherà il riferimento prima nell'oggetto maggiormente "specializzato", per poi passare alle "generalizzazioni" se quest'ultimo non è stato trovato. Questo modo di procedere si ispira quindi più alla *delegation*[33] che all'*inheritance*[34].

Per quanto riguarda il secondo tema di studio, la mia curiosità è nata dagli *snippet* di codice sotto menzionati:

```
export class PersistentSceneData extends
PersistentSingleton<PersistentSceneData>() {...}
```

```
// PersistentSingleton è una classe che ammette una sola istanza che persiste
anche alle transizioni di scena.
export function PersistentSingleton<T>() {
        abstract class PersistentSingleton extends Component {
                protected static instance: T;
                protected onLoad() {
```

[31]Versione di ECMAScript

[32]Funzioni che sono trattate come qualsiasi altra variabile. Possono essere assegnate, passate come argomento e restituite.

[33]Design pattern in cui un oggetto esternalizza alcune delle sue responsabilità a un altro oggetto.

[34]Concetto per cui una classe, detta sottoclasse, eredita proprietà e metodi da un'altra, detta superclasse.

```
                    ...
                    PersistentSingleton.instance = this as unknown as T;
                    ...
            }
        }
        return PersistentSingleton;
}
```

Dalla firma della prima classe, notiamo come in ECMAScript sia possibile estendere un valore di ritorno di una funzione, sempre che questo sia un oggetto o simile. Tale sintassi è ammessa in quanto linguaggio interpretato, per cui l'interprete traduce ed esegue la funzione e solo dopo applica il meccanismo di ereditarietà. L'altro accento va sulle motivazioni che hanno comportato questa codifica: applicare i tipi *generics*[35] ai campi statici così da beneficiare della chiarezza e robustezza della tipizzazione forte di Typescript. Di fatti, se si fosse optato direttamente per una classe PersistentSingleton generica, allora non si sarebbe potuto etichettare il campo *static* con la *wildcard* T, perché il tipo sarebbe stato confermato solo all'atto della costruzione dell'oggetto. Questa operazione non avrebbe dunque interessato la variabile statica che è associata alla classe ma non alla specifica istanza.

Il secondo *task* di rilievo a me assegnato prevedeva l'implementazione di un algoritmo per la generazione del percorso a piattaforme. Per realizzarlo, ho deciso di riusare la classe TileGenerator, presente nel vecchio gioco, per poi modificarla e adattarla alle nuove esigenze. Il flusso di generazione incomincia, dunque, all'invocazione del metodo GenerateTiles da parte del GameController, cioè il gestore del *gameplay* principale. Le istruzioni eseguite nel suo *scope* costruiscono e graficano la prima piattaforma su cui giace lo squalo alla partenza, e assieme ad essa, anche le restanti che distanziano quest'ultimo personaggio dal protagonista del gioco. Per una questione di prestazioni, le altre *tile* vengono poi istanziate ad ogni *frame* come specificato nell'Update. Nel particolare, le caselle prodotte dal primo metodo e parte delle seconde, fino ad una soglia massima indicata, sono quelle di base. A seguire, la generazione si fa più variegata, creandone della tipologia a quiz in punti precisi della sequenza e, in alternativa, in modo casuale tra le diverse categorie: con ostacoli immobili, con quelli mobili o semplicemente di base. Tutto questo finché la

[35]Concetto per cui si possono definire classi, interfacce e metodi con un tipo di dato generico, che può essere specificato al momento dell'uso, a tempo di compilazione.

seconda fase dell'algoritmo non termina con il raggiungimento di un numero limite di piattaforme. Da qui in avanti, il processo viene completato generando una *tile* per mossa del *player*, tramite la chiamata dal GameController del metodo UpdateTiles, fintantoché il contatore non uguagli il massimo numero di piattaforme per livello. Mostro, di seguito, una sezione di codice relativa alla funzione che fattorizza la meccanica di generazione della casella successiva:

```
// this.lastTile rappresenta l'ultima tile generata.
// nextTileSet rappresenta il componente TileSet del prefab istanziato.
// Il componente TileSet è pensato per comporre più tile in una piattaforma
// complessa. Non è questo il caso: le proprietà startTile ed endTile puntano
// allo stesso oggetto Tile.
private generateNextTileset(isLast: boolean): Tileset {
        let roll = math.random() >= 0.5 ? 1 : -1;
        let xRandomized = this.xTileDistance * roll;
        let startPos = this.lastTile.worldPosition;
        startPos.set(startPos.x + xRandomized, startPos.y + this.yTileDistance,
        startPos.z);
        ...
        // if(isLast) "istanzia prefab tile di transizione" else if(under threshold) "tile
        // base" else if(quiz) "quiz tile" else "random tile"
        ...
        if(roll > 0) this.lastTile.rightTile = nextTileSet.startTile;
        else this.lastTile.leftTile = nextTileSet.startTile;
        this.lastTile = nextTileSet.endTile;
        return nextTileSet;
}
```

In questo spaccato, ho voluto evidenziare la meccanica di collocamento nello spazio della nuova piattaforma così da sottolineare la struttura a lista puntata che modella il percorso. Ciascuna piattaforma ha quindi un solo riferimento funzionante alla casella alla sua sinistra o alla sua destra, mentre il rimanente non punta a nulla.

Per decorare il percorso con il logo del committente e segnalare un cambio di direzione in modo netto, nel *mockup* del gioco erano state aggiunte delle bandierine che galleggiavano nei pressi della piattaforma, al polo opposto rispetto al proseguimento del tragitto. Ho quindi inserito la meccanica in un nuovo modulo, il FlagGenerator, con questa singola responsabilità. Nel metodo OnLoad, ho in primo luogo connesso l'evento TilesLinked, appositamente predisposto nella classe TileGenerator, con la funzione di *callback* relativa. In tal modo, dall'associazione della nuova piattaforma prodotta con la precedente, nel GenerateNextTile del TileGenerator, ne consegue l'algoritmo di verifica e posizionamento dell'eventuale galleggiante. In particolare, si controlla l'avvenuto cambio di direzione tramite la presenza della piattaforma di sinistra

o di destra che inverte il cammino. Sicché, una copia del *prefab*[36] viene istanziata come figlia della piattaforma, così da disabilitare la bandiera congiuntamente alla casella, al passaggio dello squalo. Al contempo, ne si calcola la giusta posizione: se il tragitto muove verso destra, rispetto all'utente, prima di una svolta, allora la bandierina compare sulla destra e viceversa. Qui sotto le parti importanti del codice:

```
// this.currDir rappresenta la direzione attuale del percorso(RIGHT O LEFT).
// this.padding rappresenta l'offset di base dal margine della piattaforma lungo le
due direzioni.
// this.tileNormalDistance rappresenta la distanza di base lungo la x dal centro
della piattaforma.
// this.maxDeltaPadding è il massimo o il minimo offset che si può ottenere
casualmente a partire dalla tileNormalDistance
private onTilesLinked(prevTile: Tile): void {
        if(this.currDir === null) {
                this.tileNormalDistance =
                prevTile.node.getComponent(UITransform).contentSize.x / 2 *
                prevTile.node.scale.x + this.padding.x;
                this.currDir = (prevTile.leftTile !== null) ? DIR.LEFT : DIR.RIGHT;
                if(this.currDir === DIR.RIGHT) this.distance.x = -this.distance.x;
                return;
        }
        if( (prevTile.leftTile !== null && this.currDir === DIR.RIGHT) ||
            (prevTile.rightTile !== null && this.currDir === DIR.LEFT) ){
                this.distance.x = -Math.sign(this.distance.x) *
                (this.tileNormalDistance + (Math.random() < 0.5 ? -1 : 1) *
                Math.floor(Math.random() * this.maxDeltaPadding.x));
                this.currDir = (this.currDir === DIR.LEFT) ? DIR.RIGHT : DIR.LEFT;
                const flag = instantiate(this.flagPrefab);
                prevTile.node.addChild(flag);
                flag.setWorldPosition(prevTile.worldPosition.x + this.distance.x,
                prevTile.worldPosition.y, prevTile.worldPosition.z);
        }
}
```

```
private generateNextTileset(isLast: boolean): Tileset {
        ...
        this.eventTarget.emit(this.tilesLinkedEvent, this.lastTile);
        this.lastTile = nextTileSet.endTile;
        ...
}
```

Da appuntare che il primo blocco condizionale è pensato per inizializzare la variabile di stato dell'algoritmo, cioè la direzione corrente, e fissare il parametro costante, quale la distanza di base dal *pivot*[37] della piattaforma. Mentre nell'ultimo *snippet*, si vede come la *tile* passata come parametro dell'evento non è quella appena generata ma la precedente. Infatti, solo alla fine del metodo, la *reference* all'ultima piattaforma generata viene aggiornata.

[36]Oggetto di gioco preconfigurato che può essere istanziato e riutilizzato in vari punti del gioco.
[37]Punto di riferimento attorno al quale un oggetto ruota o si scala.

Successivamente, mi sono dedicato alla meccanica di comparsa e scomparsa degli ostacoli mobili. Per prima cosa, dato che gli ostacoli del gioco di partenza erano stati pensati come delle tipologie diverse di piattaforme, ho ritenuto opportuno riprogettare questi componenti tramite una classe base Tile che modella la piattaforma standard e due altre classi figlie StaticObstacleTile e DynamicObstacleTile che sovrascrivono alcune funzionalità e aggiungono nuovi attributi. Mentre gli ostacoli statici sono stati giusto rielaborati a partire dal vecchio codice, gli ostacoli dinamici sono stati implementati da zero. Entrambi condividono e ridefiniscono i metodi IsFree e Damage, ereditati dalla classe Tile, che vengono invocati dal GameController in fase di movimento del giocatore verso la piattaforma successiva, per stabilire se il passaggio è libero o meno e scatenare una reazione nel secondo caso. Alla presenza di un ostacolo dinamico, quest'ultima non consiste in un danno ma nel *reset* del *timer*. Diversamente, se l'atterraggio sulla piattaforma ha successo, allora viene disattivato il solo nodo figlio della casella con lo *sprite*[38] soprastante ma non la *tile* complessiva, che deve permanere e scomparire solo all'arrivo dello squalo. Assieme al componente grafico, si assegna un *text* vuoto al metodo Update, che invece sarebbe pensato per incrementare i *timer* e gestire la dinamica di un ostacolo attivo.

```
// this.fader.node è il nodo che rappresenta l'ostacolo sopra la piattaforma di
// base.
// this.inTransition è la flag per segnalare lo stato di transizione tra la fase di
// presenza e quella di assenza.
// this.animator è il componente che gestisce le clip animate. Lo lascio manifesto
// per giustificare la necessità di un periodo di transizione.
// this.themedSprite è il componente che gestisce le varie grafiche associate al
// personaggio. Con setSprite(0,2) si imposta l'immagine dell'ostacolo in up.
isFree(): boolean {
        if(!this.up){
                this.fader.node.active = false;
        }else if(this.inTransition){
                this.inTransition = false;
                this.animator.stop();
                this.themedSprite.setSprite(0, 2);
        }
        this.update = (dt:number) => {};
        return !this.up;
}
damage(): void {
        if(!this.fader.node.active) return;
        this.timer = 0;
```

[38]Componente di Cocos Creator che rappresenta un'immagine bidimensionale.

```
        this.update = (dt:number) => {this.elapse(dt);}
        ...
}
private elapse(dt: number){
        if(this.inTransition) return;
        this.timer += dt;
        if((this.up && this.timer > this.upTime) ||
          (!this.up && this.timer > this.downTime)){
              this.swapState();
        }
}
```

La sezione soprastante mostra come le regole rendono il *gameplay* fluido e ragionevole. In effetti, se il giocatore avvia la mossa e l'ostacolo si trova abissato, allora quest'ultimo viene direttamente disattivato in modo definitivo. Nell'altro caso, se si è in fase di discesa, allora si preferisce riportare l'ostacolo in alto ed evitare un cambio di stato. Così facendo, si risolve lo sgradevole effetto che rigettava il giocatore all'indietro anche se con un ostacolo già completamente o solo in parte scomparso. Di fatti, era possibile che a inizio spostamento del *player* l'ostacolo fosse in *up*, comportando un rimbalzo verso la piattaforma precedente, ma che si trovasse in *down* a fine remata.

L'ultimo vero elemento che caratterizza il videogioco è lo squalo che insegue il giocatore. Ho scelto di creare una nuova classe, Enemy, per questa entità che ne modellasse, in particolare, il suo movimento come reazione al verificarsi di eventi lanciati dal GameController. Quest'ultimo continua ad occuparsi, senza modifiche davvero importanti rispetto all'opera precedente, delle dinamiche della *deadzone*, che manca di un suo specifico modulo che la rappresenti. La scelta nasce dal fatto che la *deadzone* è concepita come una delle fondamenta del gioco, che non può esistere indipendentemente. Nel concreto, pertanto, lo spostamento dello squalo sostituisce l'effetto terremoto del gioco di partenza, che anticipa la scomparsa di una piattaforma e dunque l'avanzamento dell'elemento nemico. Ancora, se la guardia sulla distanza massima tra giocatore e squalo viene oltrepassata, la *deadzone* progredisce di urgenza e prima di far muovere lo squalo, viene incrementata la velocità di quest'ultimo, parificandola con quella del giocatore, per poi ripristinarla all'originale a fine mossa. Di seguito inserisco le operazioni critiche:

```
// this.currTileLifeTime è il timer per il tempo di vita della piattaforma.
// this.currTileTTL rappresenta il time to live globale di ogni piattaforma.
// this.currActiveTileBeforePlayer memorizza il numero di piattaforme attive tra
// giocatore e squalo.
// this.earlyFallEvent è l'evento che porta al cambio di velocità dello squalo.
```

```
// this.populateNextFallingTIles lancia l'evento per la scomparsa della piattaforma
// corrente e salva la successiva in una variabile d'appoggio. Con una tile a volta
// non occorreva in realtà, ma qui si lavora con degli array in continuità col
// vecchio gioco.
// this.nextPosition rappresenta la posizione della tile ancora a seguire per poter
// orientare lo squalo con this.lookToward.
// this.actualMoveTime rappresenta il tempo da impiegare per completare la
// mossa.
private updateFallingTiles(dt: number) {
        this.currTileLifetime += dt;
        if (this.currTileLifetime < this._currTileTTL
           && this.currActiveTileBeforePlayer <= this.maxActiveTileBeforePlayer)
                return;
        if(this.currActiveTileBeforePlayer > this.maxActiveTileBeforePlayer)
                this.eventTarget.emit(this.earlyFallEvent,
                                        this.player.actualJumpTime);
        this.populateNextFallingTiles();
        this.populateCurrFallingTiles();
        this.currTileLifetime = 0;
}
```

```
private moveTowards(tile: Tile) {
        if(this.moving) this.completeMovement();
        this.endTile = tile;
        const nextTile: Tile = tile.leftTile != null ? tile.leftTile : tile.rightTile;
        this.nextPosition = nextTile != null ? nextTile.worldPosition : null;
        this.performMoveTowards(tile.worldPosition);
}
private performMoveTowards(endPosition: Vec3) {
        this.startPosition = this.node.getWorldPosition();
        this.endPosition = endPosition;
        this.moveProgress = 0.0;
        this.moving = true;
}
private updateSwim(dt: number) {
        if (this.moveProgress <= this.actualMoveTime) {
                const t = this.moveProgress / this.actualMoveTime;
                const x = lerp(this.startPosition.x, this.endPosition.x, t * (2 - t));
                const y = lerp(this.startPosition.y, this.endPosition.y, t * (2 - t));
                this.node.setWorldPosition(new Vec3(x, y,
                                        this.node.worldPosition.z));
                this.moveProgress += dt;
        } else { this.completeMovement(); }
}
private completeMovement(){
        this.node.setWorldPosition(this.endPosition);
        if(this.nextPosition != null) this.lookTowards(this.nextPosition);
        this.actualMoveTime = this.moveTime;
        this.gameController.fall(this.endTile);
        this.moving = false;
}
```

Nel primo *snippet* si trova il metodo UpdateFallingTiles del GameController, che assolve il compito di determinare la piattaforma successiva da far divorare e al contempo di avviare il processo di scomparsa della corrente. L'azione nel suo complesso avviene, però, solo al verificarsi di specifiche condizioni, cioè lo scadere di un cronometro che misura il tempo di vita della casella oppure l'allontanamento eccessivo del giocatore dallo squalo. Per ambedue le

situazioni, il Controller comunica l'inizio della sparizione della piattaforma all'Enemy attraverso il lancio di un evento. Ulteriormente, il secondo scenario prevede l'emissione di un altro segnale che forza lo squalo a variare la sua velocità. La seconda sezione, invece, riassume le procedure che manovrano lo squalo. Sottolineo quindi, in questa analisi, la chiamata alla funzione CompleteMovement, che non solo si ritrova banalmente al termine del periodo di traslazione, ma piuttosto anche nell'eventualità in cui si richiede in anticipo al nemico di raggiungere la piattaforma ancora più avanti. Quest'ultimo punto rappresenta il caso del distacco oltre soglia dallo squalo da parte del protagonista. Da notare che, come piccolo appunto, la traiettoria del movimento segue una curva *ease-out*[39].

L'ultimo *task* significativo su cui ho speso diverso tempo riguardava il ridimensionamento di due nodi per la visualizzazione di immagini. Si crea quindi una nuova classe confacente SpriteResizer che dispone dei due metodi statici ResizeOnMaxSize e ResizeOnMinDistanceFromMargins. Queste due funzioni di utilità verranno poi richiamate al momento della comparsa della maschera di quiz da parte dell'entità UIQuiz già preesistente. La prima delle due viene usata per ingrandire o rimpicciolire l'immagine nella schermata con quesito e risposte, mentre la seconda fa lo stesso nel caso del *popup* ma secondo politiche diverse. In particolare:

1. ResizeOnMaxSize, che ridimensiona l'immagine in modalità *cover*, cioè moltiplicando per il fattore di scala che corrisponde al rapporto maggiore tra quello fra lunghezze e quello fra altezze del contenitore e del contenute in questo ordine;
2. ResizeOnMinDistanceFromMargins, che ridimensiona l'immagine in modalità *contain*, così da rimpicciolire o ingrandire l'immagine ma sempre nei limiti del contenitore, tenendo però in considerazione l'eventualità di una non coincidenza del centro del contenitore col centro del contenuto.

```
// contentSize rappresenta il bounding box del nodo
static resizeOnMaxScale(img: UITransform, mask: UITransform, othersToResize:
UITransform[]){
```

[39]Interpolazione utilizzata in animazione e programmazione per creare un movimento che inizia veloce e rallenta gradualmente fino a fermarsi.

```
        const originalSize = img.getComponent(Sprite).spriteFrame.originalSize;
        const scaleX = mask.contentSize.width / originalSize.width;
        const scaleY = mask.contentSize.height / originalSize.height;
        const scale = Math.max(scaleX,scaleY);
        othersToResize.forEach((o => o.setContentSize(o.contentSize.width *
                                        scale, o.contentSize.height * scale)));
        img.setContentSize(originalSize.width * scale, originalSize.height * scale);
}
static resizeOnMinDistanceFromMargins(img: UITransform, mask: UITransform,
othersToResize: UITransform[]){
        const originalSize = img.getComponent(Sprite).spriteFrame.originalSize;
        img.setContentSize(originalSize);
        const marginsImg = {
                xLeftMargin: img.node.worldPosition.x - img.contentSize.width / 2,
                yTopMargin: img.node.worldPosition.y + img.contentSize.height / 2,
                xRightMargin: img.node.worldPosition.x + img.contentSize.width / 2,
                yBottomMargin: img.node.worldPosition.y -
                                img.contentSize.height / 2
        };
        const marginsMask = {
                xLeftMargin: mask.node.worldPosition.x -
                              mask.contentSize.width / 2,
                yTopMargin: mask.node.worldPosition.y +
                             mask.contentSize.height / 2,
                xRightMargin: mask.node.worldPosition.x +
                               mask.contentSize.width / 2,
                yBottomMargin: mask.node.worldPosition.y -
                                mask.contentSize.height / 2
        };
        const delta = {
                xLeftMargins: Math.abs(marginsMask.xLeftMargin -
                                        marginsImg.xLeftMargin),
                yTopMargins: Math.abs(marginsMask.yTopMargin -
                                        marginsImg.yTopMargin),
                xRightMargins: Math.abs(marginsMask.xRightMargin -
                                         marginsImg.xRightMargin),
                yBottomMargins: Math.abs(marginsMask.yBottomMargin -
                                          marginsImg.yBottomMargin)
        };
        const deltaMin = Math.min(delta.xLeftMargins, delta.yTopMargins,
                                  delta.xRightMargins, delta.yBottomMargins);
        const keyIndex = Object.keys(delta).findIndex((k) => delta[k] ===
                                                        deltaMin);
        const mainSize = (keyIndex == 0 || keyIndex == 2) ? img.contentSize.width
                        : img.contentSize.height;
        const scale = (mainSize + deltaMin * 2) / mainSize;
        othersToResize.forEach((o => o.setContentSize(img.contentSize.width *
                                                        scale,
        img.contentSize.height * scale)));
        img.setContentSize(img.contentSize.width * scale,
                            img.contentSize.height * scale);
}
```

La prima funzione del codice in rilievo si aspetta due parametri obbligatori, quali lo UITransform[40] del nodo immagine e lo UITransform della maschera.

[40]Componente che gestisce le dimensioni dell'elemento grafico e il suo *pivot*.

Quindi recupera la dimensione originale dello *spriteFrame*[41] del nodo immagine e calcola il rapporto tra larghezze e altezze della maschera e dell'immagine. Prende il rapporto maggiore e lo usa per trovare le nuove dimensioni. Per concludere, imposta la nuova risoluzione del nodo immagine, variando conformemente le grandezze di eventuali nodi associati. Per quel che riguarda la seconda, questa si aspetta due parametri obbligatori e recupera la dimensione originale della foto come sopra. Forza lo UITransform del nodo immagine ad adeguarsi alle dimensioni originali. Vengono quindi calcolate le distanze dei margini dal centro sia per l'immagine sia per il contenitore. A seguire, misura la distanza assoluta tra i margini a coppie (es. il margine sinistro dell'immagine con quello sinistro della maschera) e prende la minima. Calcola la dimensione finale della grandezza di riferimento, *width* o *height*, cioè quella associata alla distanza minore tra margini. Infine, imposta la nuova risoluzione dell'immagine, variando conformemente le grandezze dei nodi associati, che corrispondono, nello specifico caso, all'intero *popup* padre comprendente didascalia e crediti.

Asset replacement

In questa fase dello sviluppo, mi sono preoccupato di sostituire e integrare le risorse grafiche che erano state preparate preliminarmente, in buona parte, dall' *artist* del gruppo Nicola Sampietri. Nella pratica quindi, ho scaricato le immagini in formato png da Figma e le ho caricate su Cocos, strutturando, nel mentre, una gerarchia di *directory* facilmente navigabile, atta a contenerle. Una volta importate nell'ambiente di lavoro, ho riorganizzato le schermate di menù e livelli così da attenermi il più possibile ai *mockup* su Figma e rispettare le dimensioni previste dal *design*. A tal punto, nuovi problemi sono sorti, per cui è stato necessario consultare il *team* nell'immediato: il principale problema consisteva in una alterazione della grandezza delle immagini esportate, rispetto a quella segnata sul software di grafica. A indagine svolta, ho rinvenuto i motivi del sintomo, che risiedevano nell'applicazione postuma agli *sprite* di base di effetti grafici, quali ombre essenzialmente, su Figma. Di conseguenza, la

[41]Oggetto che gestisce i dati di ritaglio e di *tiling* di un'*asset* grafico e contiene una *reference* ad una Texture2D con l'ulr al file di immagine vero e proprio.

soluzione stava in una esportazione separata della risorsa originale e dei VFX[42] ad essa apposti. Un'ulteriore indicazione fissata durante l'incontro di gruppo, era mirata a ridurre le dimensioni del prodotto attraverso la condivisione, tra più oggetti di scena, di una stessa risorsa grafica, su cui applicare, eventualmente, una colorazione differente. Per tale obiettivo, ho dovuto scaricare le figure in bianco, così da addizionare comodamente la tinta desiderata nell'*engine.*

Animation

Per uniformarsi alle altre animazioni previste, ho voluto creare un file anim[43] anche per i protagonisti dei primi due livelli, con solamente due frame: riposo e remata. Una classe appositamente pensata per la gestione delle *view* assegna all'OnEnable la *default clip* del componente Animator del *player*, in base all'attuale livello. D'altra parte, a *gameplay* avviato, l'animazione viene attivata alla chiamata dei metodi JumpTowards e BreakTowards, del modulo del Player, e fermata al termine della mossa in UpdateJump e UpdateBreak. Qui viene reimpostato pertanto lo *sprite* di riposo del giocatore. Da notare che, all'evento ThemedChangedEvent, emesso all'iniziale modifica della default clip, così come al cambio di velocità per il *dash*, si imposta la durata della clip proprio in base al tempo di spostamento, traslando anche l'ultimo *keyframe*[44] congiuntamente. Il pellicano invece, cioè l'ultimo personaggio, non possiede animazioni del genere, ma sfrutta un effetto particellare per simulare le nuvole di fumo della sua moderna nave. Qualcosa di simile ai primi due *character*, sopra menzionati, è stato realizzato per implementare le animazioni degli ostacoli dinamici, tenendo però a mente la presenza di due clip, una per la comparsa e l'altra per la scomparsa, del medesimo oggetto.

Polish

Nello stadio finale dello sviluppo, pensato per rifinire l'opera, sono stato incaricato di importare la traccia musicale di sottofondo e gli altri effetti sonori

---

[42]Acronimo di Visual Effects, a indicare l'insieme di effetti grafici applicati ad un'immagine o ad un video per migliorarlo o renderlo più realistico.

[43]Formato di file che contiene informazioni sulla *clip* animata.

[44]Punti nella linea temporale dell'animazione che sono associati a specifici valori di una proprietà di un oggetto, come la posizione. L'animazione completa viene generata calcolando per i *frame* intermedi un valore interpolato.

da associare al verificarsi di determinati eventi. Al fine di assolvere il compito, mi sono avvantaggiato del già esistente sistema di gestione dei suoni così come era stato implementato nel precedente gioco. In breve dunque, è stato sufficiente referenziare le risorse audio con un *drag and drop* sulle proprietà pubbliche del componente *Singleton* AudioData, esposte nell'*inspector* del Cocos *editor*. Inoltre, ho fatto uso di due API[45] del motore grafico per organizzare le sequenze di SFX[46] e *motion graphics* all'apparizione dei diversi menù di gioco. La prima è il sistema *scheduler*, integrato nella classe Component che permette di pianificare delle operazioni nel tempo. La seconda è chiamata Tween e fornisce un modo per definire sequenze di azioni, che possono essere concatenate e che animano, secondo delle curve di *easing,* qualsiasi proprietà di un componente.

Al fine di determinare gli elementi da migliorare nei termini di cui sopra, il *project manager* ha richiesto una *build* per effettuare un accurato test di funzionamento. A tal scopo, il *developer* mio collega ha inizialmente tentato di procedere sfruttando il servizio di *building* offerto da Cocos Creator. Con nostro dispiacere però, l'eseguibile realizzato presentava problemi in relazione ai font usati in formato ttf. Perciò, a seguito di una ricerca sulle possibili alternative, si è deciso di utilizzare il *framework open source* Tauri per impacchettare il *bundle* per il web di Cocos in un involucro tale da farne una *desktop app.* Il contenuto, quindi, viene visualizzato attraverso una *web view* che simula l'ambiente del browser senza funzionalità di *networking.*

[45]Acronimo di Application Programming Interface, a indicare l'interfaccia fornita da un sistema per interoperare con un altro.

[46]Acronimo di Sound Effects, a indicare l'insieme di effetti sonori che migliorano l'esperienza in un'opera multimediale.

CONCLUSIONE

In questa sezione conclusiva, tiro le somme dell'operato valutando i punti di forza e le debolezze delle soluzioni software utilizzate per lo sviluppo. In particolare, mi soffermo su Cocos Creator che di certo ha semplificato l'implementazione delle meccaniche grazie alle API da esso offerte e l'uso di un'architettura *component-based* ispirata al *design pattern composite.* Oltre a questo, ha fornito un ambiente visuale per l'*editing* delle scene e dei nodi, con assieme uno spazio per la gestione degli *asset*. Mentre, la maggior pecca riscontrata ha interessato il processo di *building*, mostrando poca affidabilità nella portabilità verso sistemi Windows. In parallelo, Figma mi ha permesso di condividere col *team* le illustrazioni di progetto, preziose per avere riferimenti durante l'implementazione, oltre a recuperarne le risorse grafiche non senza difficoltà. Infatti, il difetto principale riscontrato è stato, purtroppo, la sua incapacità di apporre adeguatamente gli effetti visivi sugli *asset* importati, portando ad una inconsistenza nelle dimensioni tra le immagini esportate e quelle visualizzate nel suo *workspace*.

Pongo ora l'accento sulle conoscenze e le competenze acquisite. Ebbene, posso di certo affermare di aver esplorato un ambiente di sviluppo a me prima sconosciuto, Cocos Creator, a punto tale da sentirmi capace di reimpiegarlo autonomamente in altri progetti. Tant'è vero che effettivamente ho intenzione di optare per l'uso di questa tecnologia anche per il mio lavoro di tesi. Congiuntamente, ho avuto la possibilità di approfondire e consolidare meccanismi e principi già trattati o almeno presentati nei corsi universitari che attualmente frequento o antecedenti. In realtà, questo percorso non ha solo potenziato le mie *hard skill*, altresì le *soft*, quali collaborazione in *team,* gestione della propria produttività e capacità di chiedere aiuto senza indugio.

Oltre ad essere formativa, l'esperienza mi ha avvicinato al mondo del lavoro e, in particolare, alla filiera videoludica italiana così che ne prendessi maggiore consapevolezza. Infatti, ho potuto operare su una buona fetta del processo di sviluppo, a partire da un semilavorato fino alla creazione di una *build* di produzione che effettivamente è stata, con mia soddisfazione, accettata dal cliente, con l'istallazione dell'opera che verrà effettuata in data, bensì, ancora ignota sia a me sia all'azienda.

Per coronare l'analisi, concludo col dire che il percorso ha ravvivato le mie future aspirazioni, rassicurandomi sulla direzione intrapresa per l'ipotetica laurea magistrale o, comunque, ha comprovato la mia reale passione nel campo.

SITOGRAFIA

- Cocos. (n.d.). *Cocos - The world's top 2D&3D engine, game / smart cockpit / AR/VR / virtual character / education*. Cocos. Ultimo accesso: 5 maggio 2025, https://www.cocos.com/en.
- IIDEA. (n.d.). *Home - IIDEA*. IIDEA. Ultimo accesso: 5 maggio 2025, https://iideassociation.com/.
- IIDEA. (2025). *Videogiochi nel 2024: consumi per quasi 2,4 miliardi di euro e 14 milioni di appassionati. Cresce il Made in Italy con un +25% di imprese, +17% di occupati e +36% di fatturato*. IIDEA. Ultimo accesso: 5 maggio 2025, https://iideassociation.com/videogiochi-nel-2024-consumi-per-quasi-24-miliardi-di-euro-e-14-milioni-di-appassionati-cresce-il-made-in-italy-con-un-25-di-imprese-17-di-occupati-e-36-di-fatturato/.
- Indici Opponibili. (n.d.). *Indici Opponibili*. Indici Opponibili. Ultimo accesso: 5 maggio 2025, https://www.indiciopponibili.com/.
- IPID. (n.d.). *Svilupparty*. IPID. Ultimo accesso: 5 maggio 2025, https://www.ipid.dev/svilupparty/.
- Cotoletta8bit. (n.d.). *Cotoletta8bit*. Cotoletta8bit. Ultimo accesso: 5 maggio 2025, https://www.cotoletta8bit.it/.
- First Playable. (n.d.). *First Playable – Where games grow*. First Playable. Ultimo accesso: 5 maggio 2025, https://firstplayable.it/.
- Wikipedia. (n.d.). *Entity component system - Wikipedia*. Wikipedia. Ultimo accesso: 5 maggio 2025, https://en.m.wikipedia.org/wiki/Entity_component_system.
- Discord. (n.d.). *Discord - Chat di gruppo super mega iper ultra divertente*. Discord. Ultimo accesso: 5 maggio 2025, https://discord.com/.
- Wikipedia. (n.d.). *Discord - Wikipedia*. Wikipedia. Ultimo accesso: 5 maggio 2025, https://en.wikipedia.org/wiki/Discord.
- Notion. (n.d.). *Notion Desktop App for Mac & Windows*. Notion. Ultimo accesso: 5 maggio 2025, https://www.notion.com/desktop.
- Wikipedia. (n.d.). *Notion (productivity software) - Wikipedia*. Wikipedia. Ultimo accesso: 5 maggio 2025, https://en.m.wikipedia.org/wiki/Notion_(productivity_software)).
- Figma. (n.d.). *Figma: The Collaborative Interface Design Tool*. Figma. Ultimo accesso: 5 maggio 2025, https://www.figma.com/
- Wikipedia. (n.d.).*Figma - Wikipedia*. Wikipedia. Ultimo accesso: 5 maggio 2025, https://en.wikipedia.org/wiki/Figma.
- MDN Web Docs. (n.d.). *Object prototypes - Learn web development | MDN*. MDN Web Docs. Ultimo accesso: 5 maggio 2025, https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Advanced_JavaScript_objects/Object_prototypes.
- MDN Web Docs. (2024). *First-class Function - MDN Web Docs Glossary: Definitions of Web-related terms | MD*. MDN Web Docs. Ultimo accesso: 5 maggio 2025, https://developer.mozilla.org/en-US/docs/Glossary/First-class_Function.
- Cocos. (n.d.). *GitHub - cocos/cocos-engine: Cocos simplifies game creation and distribution with Cocos Creator, a free, open-source, cross-platform game engine. Empowering millions of developers to create high-performance, engaging 2D/3D games and instant web entertainment.* GitHub. Ultimo accesso: 5 maggio 2025, https://github.com/cocos/cocos-engine.
- DeepWiki. (n.d.). *Native Development | cocos/cocos-engine | DeepWiki*. DeepWiki. Ultimo accesso: 5 maggio 2025, https://deepwiki.com/cocos/cocos-engine/6-native-development.
- Cocos. (n.d.). *Life Cycle Callbacks | Cocos Creator*. Cocos. Ultimo accesso: 5 maggio 2025, https://docs.cocos.com/creator/3.8/manual/en/scripting/life-cycle-callbacks.html.

- DeepWiki. (n.d.). *Engine Core | cocos/cocos-engine | DeepWiki*. DeepWiki. Ultimo accesso: 5 maggio 2025, https://deepwiki.com/cocos/cocos-engine/2-engine-core.